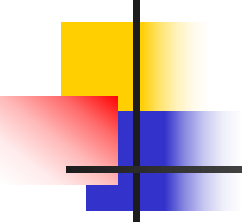


第5章 循环神经网络及应用



湖北大学 计算机学院
王雷春



目录

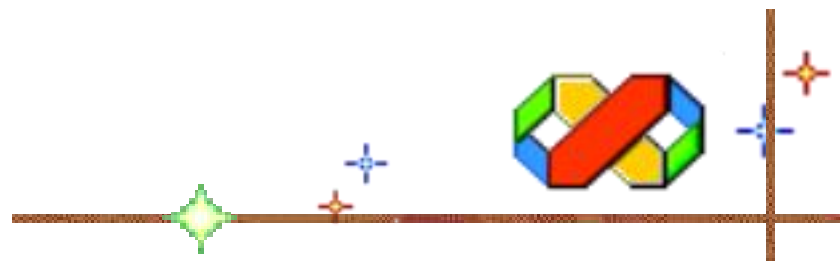
5.1 概述

5.2 RNN分类

5.3 标准RNN

5.4 经典RNN及使用方法

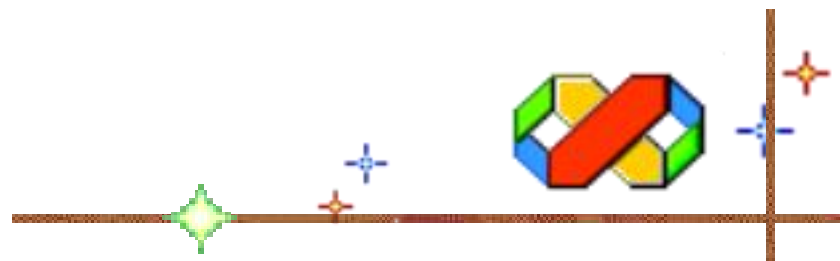
5.5 RNN应用及案例

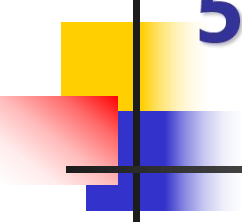


5.4 经典RNN及使用方法

5.4.1 经典RNN

5.4.2 使用方法





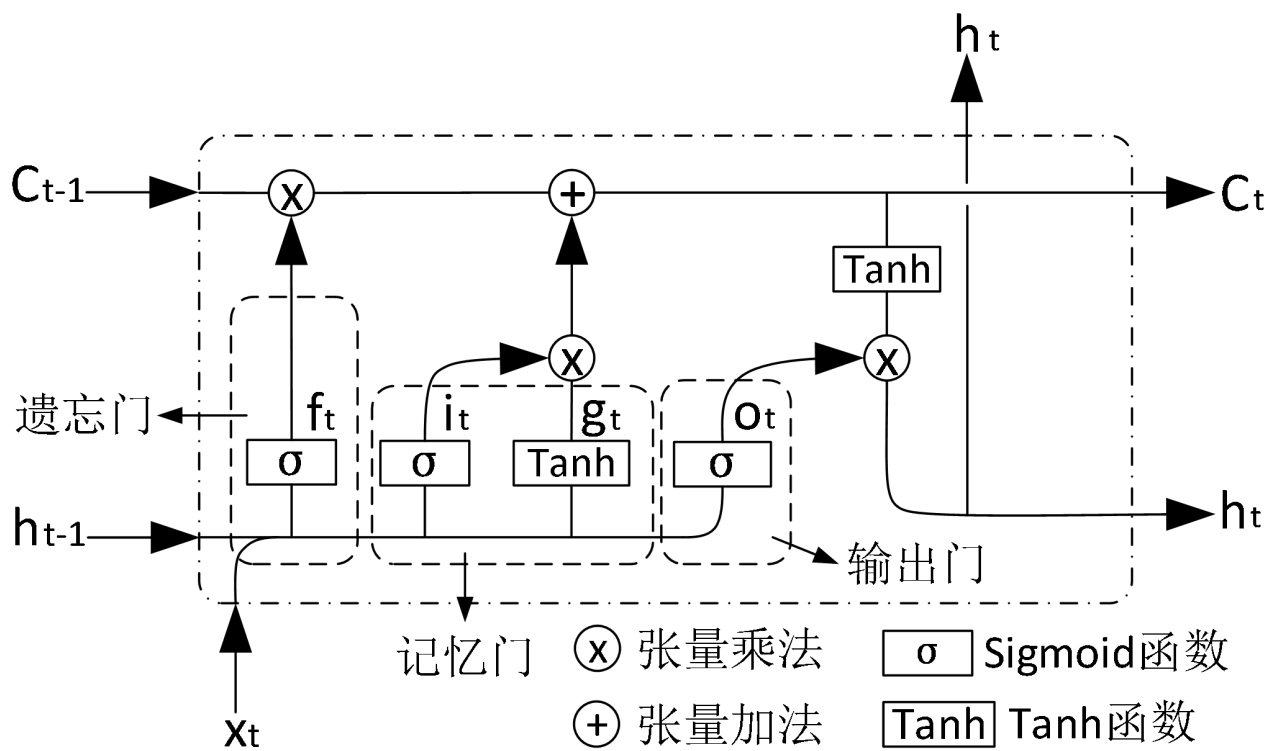
5.4.1.1.长短期记忆网络 (LSTM)

1. 什么是LSTM

- ◆ 长短期记忆网络 (Long Short-Term Memory, LSTM) 是一种特殊的循环神经网络 (RNN)，是为了解决传统RNN存在的长期依赖问题（很难学习到跨度较大的时间步之间的依赖关系）及梯度消失、梯度爆炸问题由Hochreiter 和 Schmidhuber在1997年提出。
- ◆ LSTM通过引入一个精巧的门控机制和一条独立的细胞状态线，让模型能够自主决定哪些信息需要记住、哪些需要遗忘、哪些需要输出，从而实现了长期信息的高效存储和利用。
- ◆ LSTM 是序列建模领域最经典的基线模型之一，尽管后续出现了GRU（门控循环单元，结构更简化）和Transformer（基于注意力机制），但LSTM因其可解释性和良好的实践效果仍被广泛使用。

5.4.1.1.长短期记忆网络 (LSTM)

2. LSTM结构



5.4.1.1.长短期记忆网络 (LSTM)

2. LSTM结构

(1) 遗忘门 f_t 。遗忘门决定哪些信息从细胞状态中丢弃。它接收上一时刻的隐藏状态和当前输入，通过 $Sigmoid$ 函数输出 0 到 1 之间的值，接近 0 表示“完全遗忘”，接近 1 表示“完全保留”。 f_t 计算方法如下：

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

其中， W_f 和 b_f 表示遗忘门的权重和偏置； h_{t-1} 表示上一时刻的隐藏状态； x_t 表示当前时刻的输入。

(2) 输入门 i_t 。控制新信息写入细胞状态的程度。由两部分组成： $Sigmoid$ 层决定更新位置(i_t)； $Tanh$ 层生成候选值 $\tilde{C}_t$ ，两者相乘后更新细胞状态。 i_t 和 $\tilde{C}_t$ 的计算方法如下：

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

其中， W_i 、 W_h 、 b_i 和 b_c 表示权重和偏置。

(3) 细胞状态更新 C_t 。 C_t 是结合遗忘门和输入门的结果，表示当前时刻的记忆单元信息，计算方法如下：

$$C_t = f_t \odot c_{t-1} + i_t \odot \tilde{C}_t$$

其中， $\odot$ 表示 Hadamard 积（逐元素相乘）。

5.4.1.1.长短期记忆网络 (LSTM)

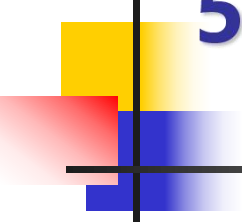
2. LSTM结构

(4) 输出门。决定细胞状态的哪部分作为当前隐藏状态输出。*Sigmoid*层决定输出部分，*Tanh*层对细胞状态进行压缩，两者相乘得到最终输出 h_t 。

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

其中， W_o 、 b_i 和 b_c 表示权重和偏置； σ 表示*Sigmoid*层。

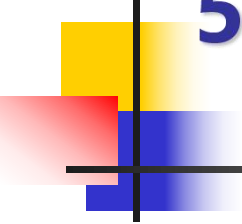


5.4.1.1.长短期记忆网络 (LSTM)

3. 优势和局限

(1)优势

- ◆ 解决长期依赖：通过门控机制和细胞状态，有效克服了梯度消失问题，能捕捉到上百个时间步的依赖关系。
- ◆ 性能出色：在许多序列建模任务上（如早期机器翻译、语音识别、文本生成），LSTM相较于传统RNN有显著提升。
- ◆ 控制信息流动：门控机制提供了精细的信息控制能力，使模型更灵活。

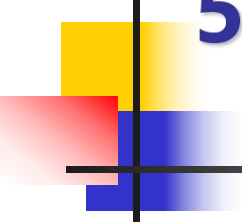


5.4.1.1.长短期记忆网络 (LSTM)

3. 优势和局限

(2)局限

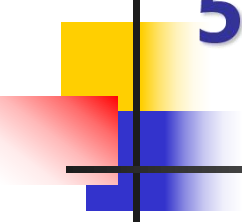
- ◆ 计算复杂度高：相比简单的RNN，LSTM有更多的参数和门控计算，训练和推理速度较慢。
- ◆ 难以完全并行：LSTM作为RNN的一种，其循环本质限制了其在序列维度上的并行计算能力，处理长序列时效率不高。
- ◆ 对极长序列仍有挑战：尽管改进很大，但对于非常非常长的序列（如数万时间步），LSTM仍然可能出现梯度消失或记忆衰减。
- ◆ 可解释性差：门控机制的内部运作相对复杂，不容易直观解释模型学到了什么。



5.4.1.1.长短期记忆网络 (LSTM)

4. 应用场景

- ◆ (1)时序数据预测（如股票、天气、传感器数据）：这类数据通常不需要GPT级别的上下文长度，LSTM更轻量、更成熟。
- ◆ (2)小数据集：当数据量有限时，LSTM的性能往往优于需要海量数据的Transformer。
- ◆ (3)计算资源受限：在边缘设备、移动设备上，LSTM的推理成本较低。



5.4.1.2.门控循环单元 (GRU)

1. 什么是GRU

- ◆ 门控循环单元 (Gated Recurrent Unit, GRU) 是另一种常用的循环神经网络，可以看作是 LSTM 的简化版。
- ◆ 它由Cho等人在2014年提出，同样为了解决传统 RNN 的梯度消失问题和长期依赖问题，但结构更精简、参数更少，训练速度通常更快。

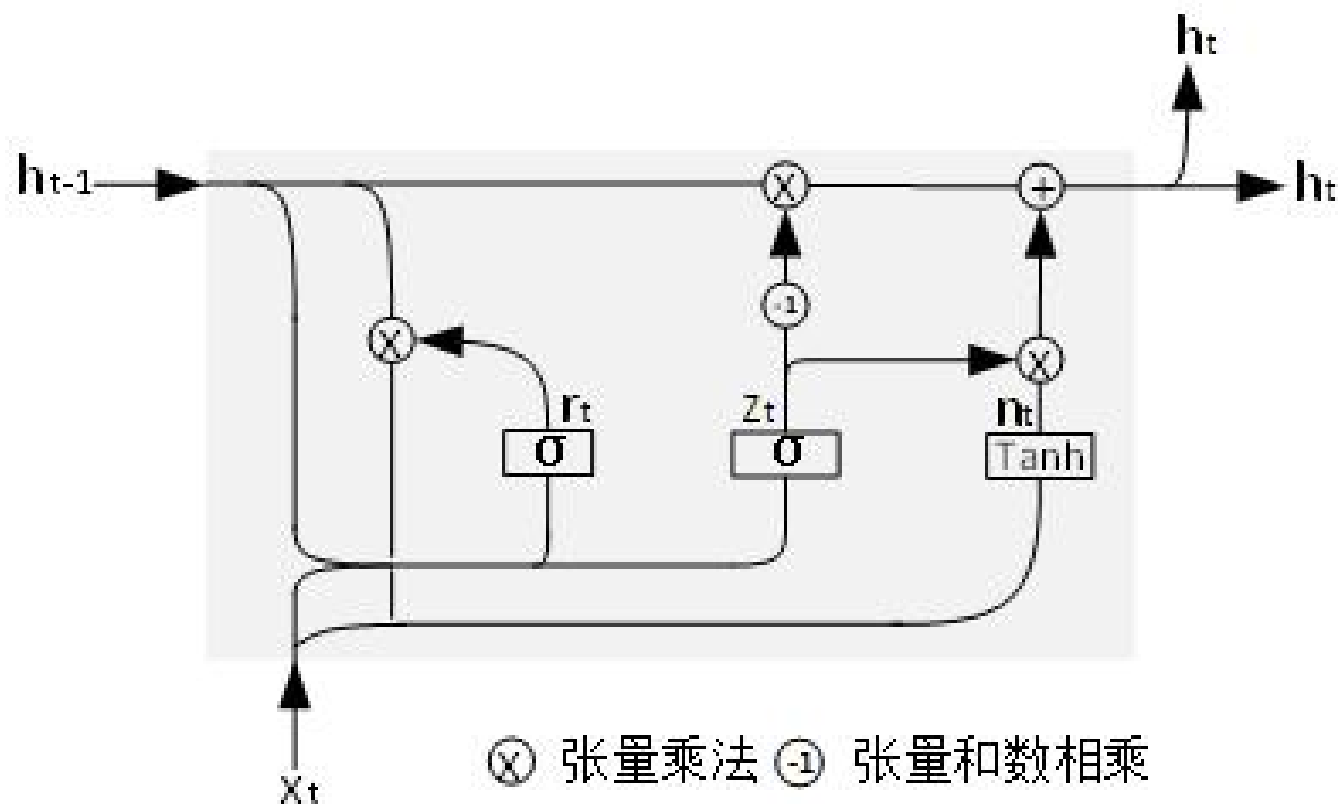
5.4.1.2. 门控循环单元 (GRU)

1. 什么是GRU

- ◆ GRU与 LSTM的核心区别在于结构复杂度和计算效率，LSTM 更强大但更复杂，GRU 更简洁且更快。
 - ◆ 门控数量：LSTM 使用3个门（遗忘门、输入门、输出门）；GRU 使用2个门（更新门、重置门）。
 - ◆ 记忆单元：LSTM 拥有独立的细胞状态和隐藏状态；GRU 没有细胞状态，仅保留隐藏状态。
 - ◆ 计算效率：GRU 结构更简单，参数量更少，因此训练速度更快，推理开销更小。
 - ◆ 性能表现：在中小规模数据集上，GRU 通常与 LSTM 表现相当甚至更好；在需要极精细记忆控制的极长序列任务上，LSTM 可能略有优势。

5.4.1.2. 门控循环单元 (GRU)

2. GRU结构



5.4.1.2. 门控循环单元 (GRU)

2. GRU结构

(1) 更新门(z_t): 决定保留多少历史信息 h_{t-1} 与加入多少新信息。输出为 0~1 的向量, 值越大, 表示保留的旧信息越多。 z_t 的计算方法如下:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

其中, W_z 和 b_z 分别是更新门的权重矩阵和偏置, 是可学习的参数; $[h_{t-1}, x_t]$ 表示 h_{t-1} 和 x_t 的拼接(维度合并); σ 为 Sigmoid 激活函数。

(2) 重置门(r_t): 决定忽略多少过去的信息, 用于计算候选状态。输出为 0~1 的向量, 值越小, 表示“重置”得越彻底。 r_t 的计算方法如下:

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

其中, W_r 和 b_r 分别是重置门的权重矩阵和偏置, 是可学习的参数。

(3) 候选隐藏状态 $\tilde{h}_t$: 基于当前输入和重置后的历史信息生成新内容。 $\tilde{h}_t$ 的计算方法如下:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

其中, $\odot$ 表示元素级乘法; $r_t \odot h_{t-1}$ 表示仅保留重置门允许的历史信息; W_h 和 b_h 为候选隐藏状态的权重和偏置。

5.4.1.2. 门控循环单元 (GRU)

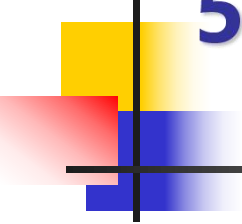
2. GRU结构

(4) 最终隐藏状态 h_t ：结合旧状态与候选状态得到当前输。 h_t 的计算方法如下：

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

如果 $z_t \approx 1$ ： $h_t \approx \tilde{h}_t$ ，几乎完全采用候选状态（类似“更新”），忽略旧状态。

如果 $z_t \approx 0$ ： $h_t \approx h_{t-1}$ ，几乎完全保留旧状态（类似“遗忘”），忽略新信息。

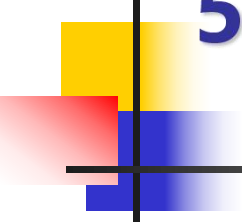


5.4.1.2.门控循环单元 (GRU)

3. 优势和局限

(1)优势

- ◆ 参数更少：相比 LSTM 少了 1 个门和 1 个状态向量，参数量减少约 25%。
- ◆ 训练更快：结构简单，矩阵运算量小，收敛速度通常优于 LSTM。
- ◆ 不易过拟合：参数少意味着在小数据集上过拟合风险更低。
- ◆ 性能不输 LSTM：在许多任务上（尤其是中小规模数据），GRU 的表现与 LSTM 相当甚至更好（如情感分析、文本分类、时间序列预测）。
- ◆ 可解释性稍好：结构更简洁，更容易理解信息流动。

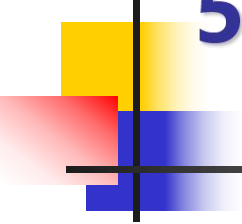


5.4.1.2. 门控循环单元 (GRU)

3. 优势和局限

(2) 劣势

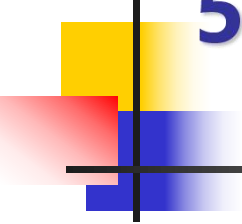
- ◆ 表达能力上限略低：因为没有独立的细胞状态，在需要极其精细的记忆控制任务上（如某些机器翻译、语音生成），LSTM 可能仍有微弱优势。
- ◆ 无法完全替代 LSTM：对于极长序列（如数千步）或需要同时维护多个记忆模式的任务，LSTM 的独立细胞状态可能更有优势。



5.4.1.2.门控循环单元 (GRU)

4. 应用场景

- ◆ (1)中小规模序列建模：如情感分析、主题分类、新闻分类等。
- ◆ (2)时间序列预测：如股票价格预测、电力负荷预测、天气预测、传感器数据分析等。
- ◆ (3)资源受限环境：如移动端 App 中的实时推理、边缘设备（如智能家居）、嵌入式系统等。
- ◆ (4)实时流式处理：如语音端点检测、手势识别、金融高频交易中的实时预测等。
- ◆ (5)快速原型验证：如学术研究中的基线模型、新思路的快速验证等。



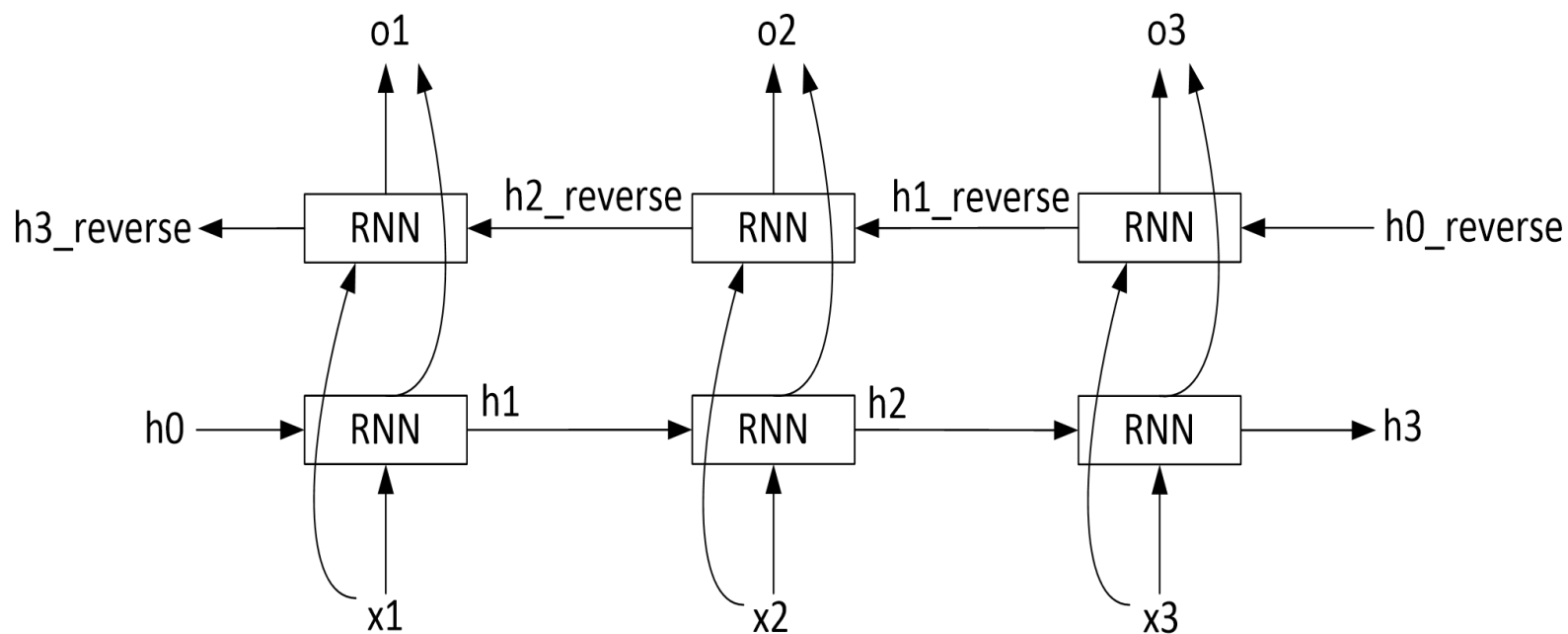
5.4.1.3.双向循环神经网络(BiRNN)

1. 什么是BiRNN

- ◆ 双向循环神经网络 (Bidirectional RNN, BiRNN) 是一种通过同时利用序列的前向和后向信息来增强模型理解的网络结构，是对标准 RNN (包括 LSTM、GRU) 的扩展。
- ◆ BiRNN是一种由两个独立的循环神经网络 (RNN、LSTM或GRU) 组成的结构：一个按时间正序处理序列 (前向 RNN)，另一个按时间逆序处理序列 (后向 RNN)。每个时间步的输出由两个方向的隐藏状态拼接 (或求和等组合方式) 得到，从而让模型同时捕捉该时刻的过去和未来上下文信息。

5.4.1.3.双向循环神经网络(BiRNN)

2. BiRNN结构



5.4.1.3.双向循环神经网络(BiRNN)

2. BiRNN结构

对于一个时间步长度为 T 的输入序列 $x_t = \{x_1, x_2, \dots, x_T\}$:

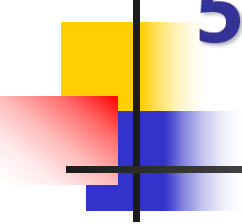
- 前向RNN: 按时间正序计算, 得到隐藏状态 $\{\vec{h}_1, \vec{h}_2, \dots, \vec{h}_T\}$ 。
- 后向RNN: 按时间正序计算, 得到隐藏状态 $\{\overleftarrow{h}_1, \overleftarrow{h}_2, \dots, \overleftarrow{h}_T\}$ 。

每个时间步 t 的最终输出为两个隐藏状态的组合:

$$h_t = [\vec{h}_t; \overleftarrow{h}_t] \text{ (拼接)}$$

或

$$h_t = [\overleftarrow{h}_t; \vec{h}_t] \text{ (拼接)}。$$



5.4.1.3.双向循环神经网络(BiRNN)

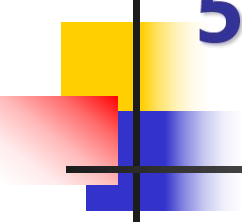
3. 优势和局限

(1)优势

- ◆ 上下文信息更丰富：在需要理解整个序列的任务中，效果显著优于单向 RNN。
- ◆ 特征表达能力更强：每个位置的特征融合了前后信息，表述更准确。
- ◆ 灵活适配：可以将内部的 RNN 单元换成 LSTM 或 GRU，得到 BiLSTM 或 BiGRU，从而兼具长期记忆能力。

(2)局限

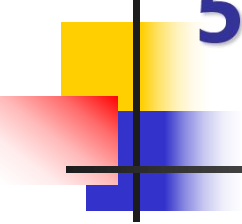
- ◆ 无法实时处理：必须事先拿到完整的输入序列才能计算反向部分，不适合流式数据（如语音实时识别、在线交易预测）。
- ◆ 参数量翻倍：训练和推理成本高于单向 RNN。
- ◆ 对极长序列仍有挑战：即使双向，RNN 的梯度问题依然存在。



5.4.1.3.双向循环神经网络(BiRNN)

4. 应用场景

- ◆ 自然语言处理：如文本分类、情感分析、命名实体识别。
- ◆ 语音识别：如语音转文字（离线版）。
- ◆ 机器翻译：如编码器（Encoder）部分。
- ◆ 生物信息学：如蛋白质结构预测、基因序列分析。
- ◆ 手写识别：如离线手写文字识别。



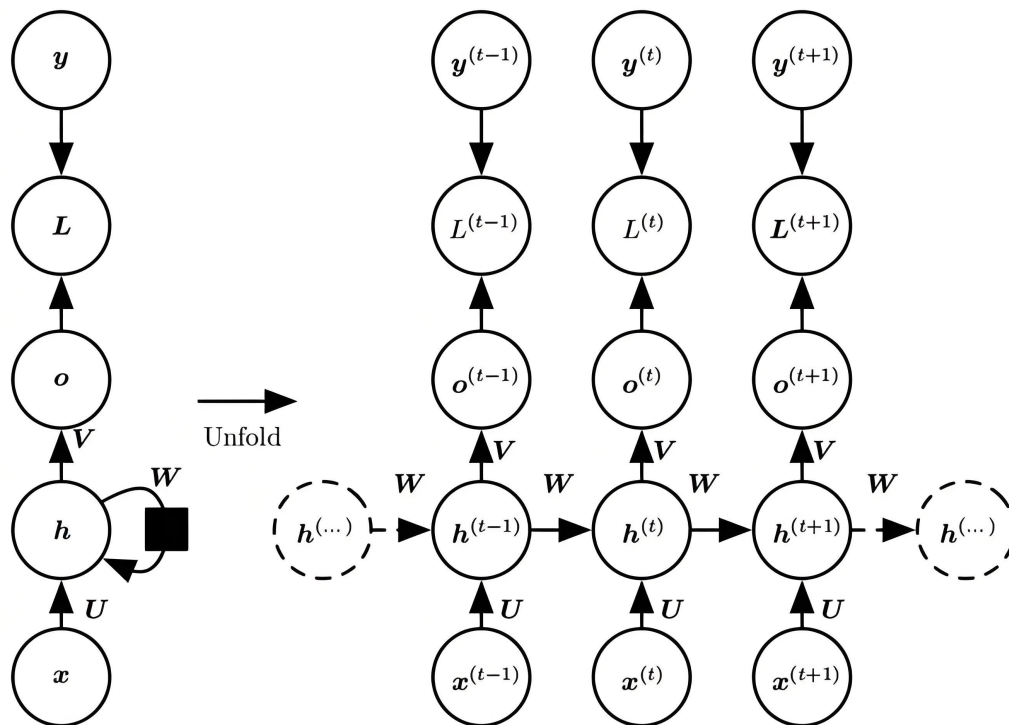
5.4.1.4.深度循环神经网络(Deep RNN)

1. 什么是Deep RNN

- ◆ 深度循环神经网络 (Deep RNN) 指的是在时间维度上展开，同时在空间维度（层数）上堆叠多个循环层（如RNN、LSTM、GRU单元）的循环神经网络，它的核心思想是通过增加网络深度，提升模型的表达能力和抽象层次。
- ◆ 普通 RNN只有一层循环，在时间维度上展开（横向），信息随时间步传递。Deep RNN在每一时间步上还有多个 RNN 层纵向堆叠，上一层的隐藏状态作为下一层的输入，形成层次化的特征提取。

5.4.1.4.深度循环神经网络(Deep RNN)

2. Deep RNN结构



5.4.1.4.深度循环神经网络(Deep RNN)

2. Deep RNN结构

假设 Deep RNN 有 L 个循环层，处理时间步长度为 T 的输入序列 $x_t = \{x_1, x_2, \dots, x_T\}$ 。

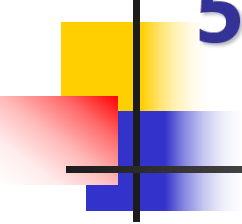
第 1 层：接收原始输入 x_t 和上一时刻自己的隐藏状态 $h_{t-1}^{(1)}$ ，输出当前时刻的隐藏状态 $h_t^{(1)}$ 。

第 2 层：接收第 1 层的输出 $h_t^{(1)}$ 和上一时刻自己的隐藏状态 $h_{t-1}^{(2)}$ ，输出 $h_t^{(2)}$ 。

...

第 L 层（顶层）：接收第 $L - 1$ 层的输出 $h_t^{(L-1)}$ 和上一时刻自己的隐藏状态 $h_{t-1}^{(L)}$ ，输出 $h_t^{(L)}$ 。最终送入输出层（如全连接+softmax）。

每个循环层内可以是基础 RNN 或 LSTM、GRU 单元。



5.4.1.4.深度循环神经网络(Deep RNN)

3. 优势和局限

(1)优势

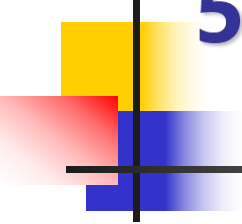
- ◆ 更强的表达能力：深度网络能学习更复杂、更抽象的模式。
- ◆ 更好的性能：在足够大的数据集上，深层 RNN 通常比浅层 RNN 效果好。
- ◆ 层次化特征学习：底层可能学习到短时依赖（如音素），高层学习到长时依赖（如短语结构）。

5.4.1.4.深度循环神经网络(Deep RNN)

3. 优势和局限

(2)局限

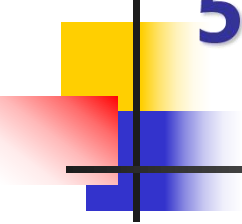
- ◆ 梯度问题更严重：深度 + 时间展开 = 非常深的计算图，梯度消失/爆炸风险成倍增加。
- ◆ 计算成本高：参数量随层数线性增长，训练时间显著增加。
- ◆ 过拟合风险：深度模型更容易记住训练数据，需要 dropout 等正则化手段。
- ◆ 并行化困难：RNN 的循环本质使其难以在时间维度上并行，深度进一步加剧了计算负担。
- ◆ 收益递减：超过一定层数（通常 3~5 层）后，继续加深带来的性能提升很小，但成本飙升。



5.4.1.4.深度循环神经网络(Deep RNN)

4. 应用场景

- ◆ 语音识别：如声学建模；
- ◆ 机器翻译：如编码器-解码器；
- ◆ 文本分类：如长文档分类；
- ◆ 手写识别：如离线手写文字识别。

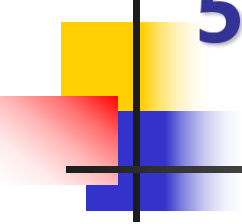


5.4.2.使用方法

1. PyTorch中RNN模块

(1)三大核心模块

- ◆ nn.RNN: 基础循环神经网络, 结构最简单, 参数最少。处理短序列, 或作为快速验证的基线模型。
- ◆ nn.LSTM: 长短期记忆网络, 包含独立的“细胞状态”, 通过门控结构有效解决长期依赖问题, 性能强大但参数多。需要捕捉长距离依赖的复杂任务, 如机器翻译、时间序列预测。
- ◆ nn.GRU: 门控循环单元, LSTM 的简化版, 将三个门简化为两个, 性能与 LSTM 接近, 但参数更少、训练更快。计算资源受限或追求训练速度的场景。

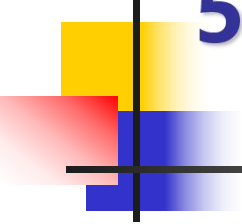


5.4.2.使用方法

1. PyTorch中RNN模块

(2)核心参数详解 (以 LSTM 为例)

- ◆ 在实例化这些模块时，最常用的参数是下面这些，它们为 RNN 的变体和深度模型提供了高度一致的配置方式。
- ◆ input_size: 输入特征的数量。
- ◆ hidden_size: 隐藏状态的特征数量。
- ◆ num_layers: RNN的层数（深度），>1 时为深度 RNN，默认值为1。
- ◆ batch_first: 指定输入/输出张量的形状，默认值为False。
- ◆ dropout: 层间的 Dropout 比率，防止过拟合，默认值为0。
- ◆ bidirectional: 是否为双向 RNN False bidirectional=True。
- ◆ bias: 是否包含偏置项，默认值为True。



5.4.2.使用方法

2. 案例

【例5-2】正弦波预测。

- ◆ A.导入必要库。
- ◆ 包括torch、torch.nn、numpy、matplotlib.pyplot和和torch.utils.data等。
- ◆ 为了能正确显示汉字和负号，对它们进行了相应的设置，代码如下：
- ◆ `plt.rcParams['font.sans-serif'] = ['Microsoft YaHei']` # 设置字体为'SimHei'
- ◆ `plt.rcParams['axes.unicode_minus'] = False` # 正确显示负号

5.4.2.使用方法

2. 案例

【例5-2】正弦波预测。

- ◆ B.生成数据。生成一段正弦波，并构造监督学习样本（ X =前lookback个点， y =下一个点）。
- ◆ 使用numpy中的函数linspace生成一段正弦波；
- ◆ 根据生成的正弦波数据构造数据集；
- ◆ 将数据集划分为训练集和测试集；
- ◆ 将训练集和测试集转换为张量。
- ◆ C.创建 DataLoader
- ◆ 使用TensorDataset将数据打包；
- ◆ 使用DataLoader对打包后的进行批量加载。

5.4.2.使用方法

2. 案例

【例5-2】正弦波预测。

- ◆ D.定义 GRU模型。使用PyTorch中的nn.GRU构建GRU模型。
- ◆ E.训练模型。损失函数使用MSE函数，优化器使用Adam，学习率初始值设为0.001，训练轮数设为50。
- ◆ F.评估。
 - ◆ 在测试集上计算预测值和真实值之间的MSE误差；
 - ◆ 输出MSE误差。
- ◆ G.结果可视化。
 - ◆ 在坐标图中绘制前200个测试样本的预测和真实；
 - ◆ 在坐标图中绘制绘制训练损失曲线。

5.4.2.使用方法

2. 案例

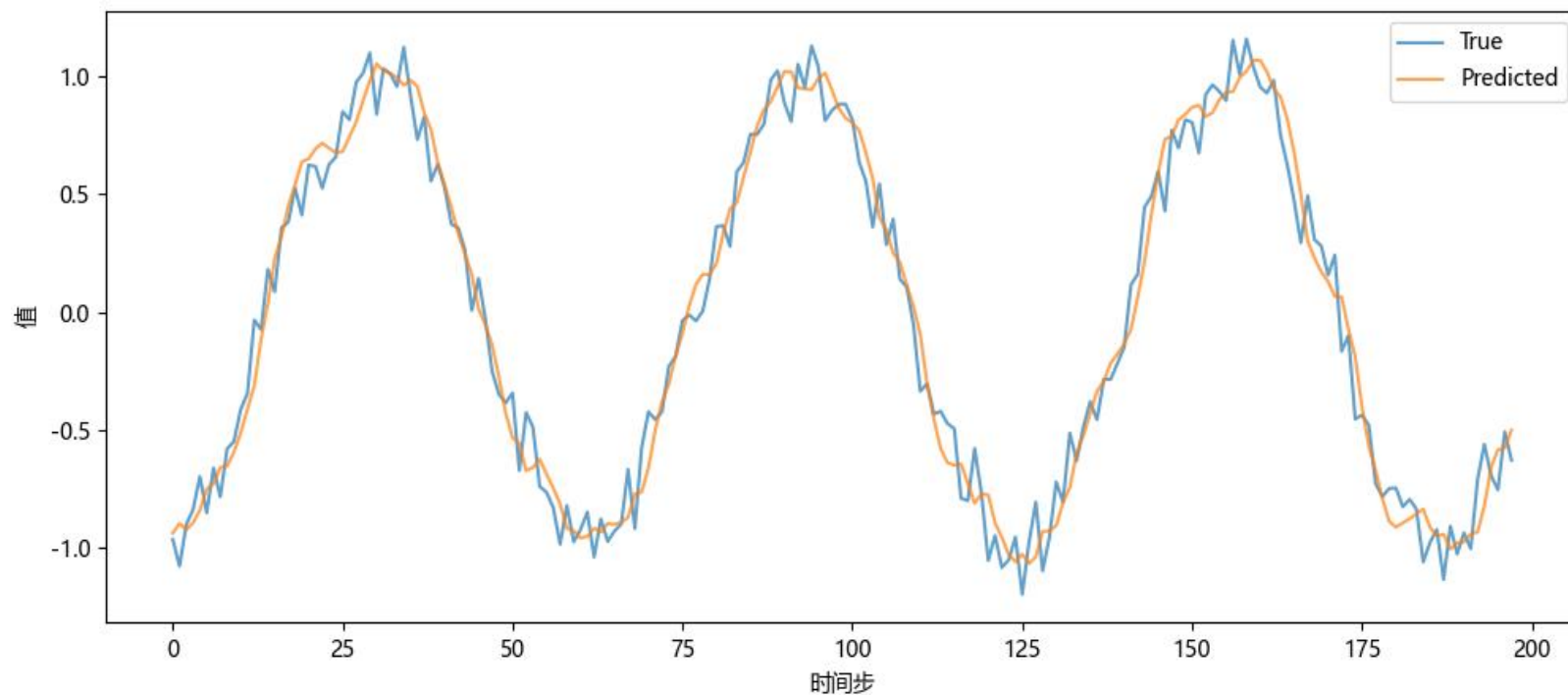
【例5-2】 正弦波预测。

运行结果

- ◆ 训练集大小: 792, 测试集大小: 198
- ◆ GRUPredictor(
 - ◆ (gru): GRU(1, 32, num_layers=2, batch_first=True)
 - ◆ (fc): Linear(in_features=32, out_features=1, bias=True)
 - ◆)
- ◆ Epoch [10/50], Loss: 0.015101
- ◆ Epoch [20/50], Loss: 0.015349
- ◆ Epoch [30/50], Loss: 0.014860
- ◆ 测试集 MSE: 0.016324

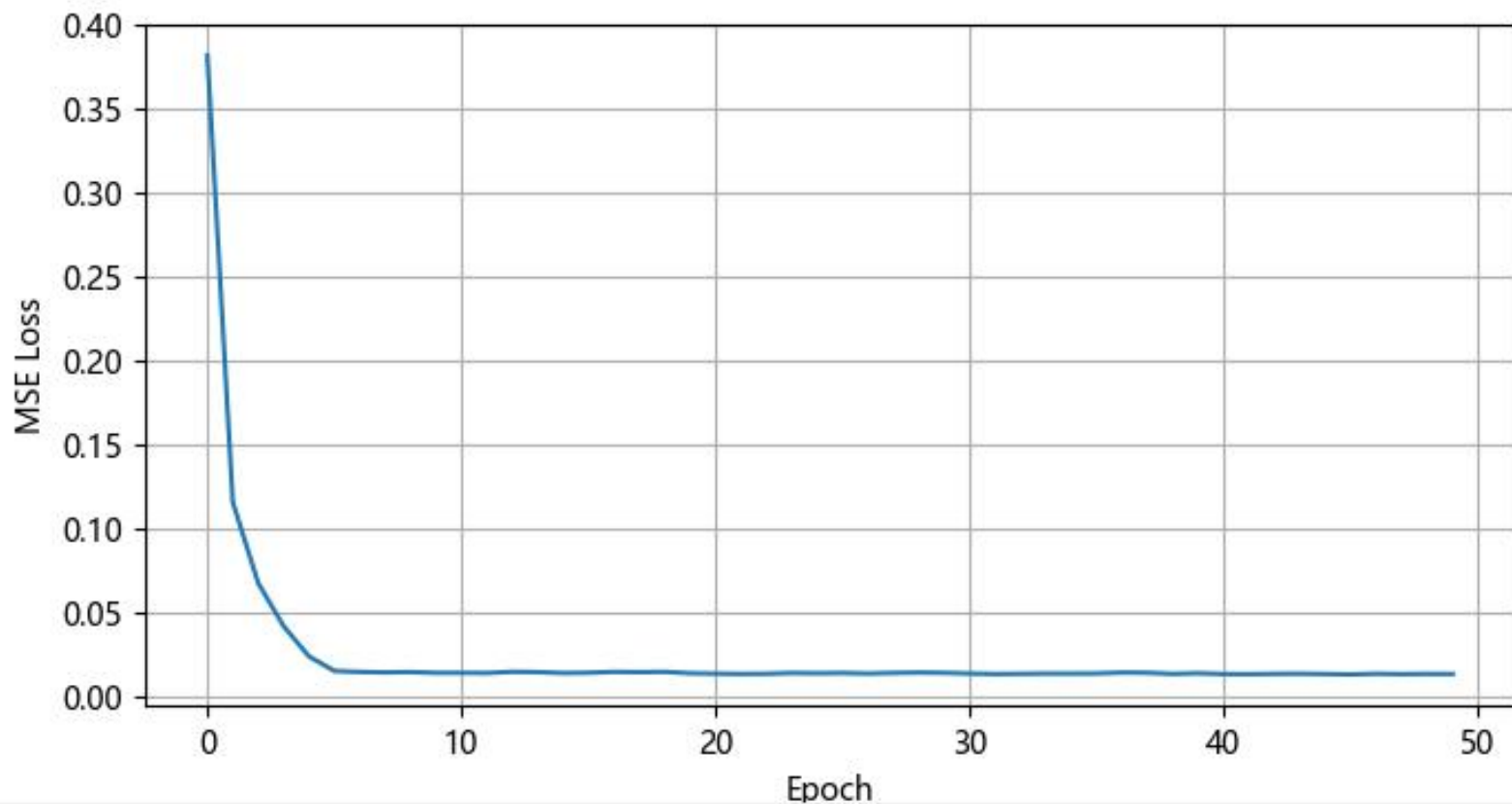
5.4.2.使用方法

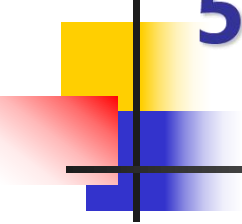
2. 案例



5.4.2.使用方法

2. 案例

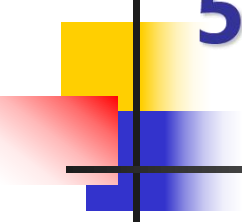




5.5 应用案例

1. 什么是文本分类

- ◆ 文本分类是自然语言处理（NLP）中最基础、最核心的任务之一，其本质是利用计算机自动为一段文本分配一个或多个预定义类别的过程。
- ◆ 简单的说，就是让机器学会“阅读”文本，然后判断它属于哪个或哪几个预先设定好的“门类”。

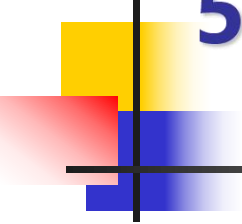


5.5 应用案例

1. 什么是文本分类

文本分类包括三个核心要素：

- ◆ (1)待分类的文本：系统的输入，可以是任何形式的文本，如：短文本（如一条微博、商品评论、新闻标题等），长文本（如一篇新闻报道、学术论文、法律文书等）。
- ◆ (2)预定义的类别：系统的输出目标，必须是一个互斥或非互斥的集合，如新闻分类中的"体育"、"财经"、"军事"（互斥）；一篇文章可以同时被标记为"科技"和"创新"（非互斥）。
- ◆ (3)分类规则或模型：系统的核心"大脑"，定义了从文本到类别的映射关系，包括：规则方法、统计/机器学习方法和深度学习方法等。



5.5 应用案例

2. 类型

文本分类包括三个核心要素：

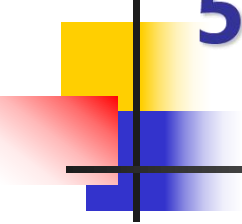
- ◆ (1)二分类：是最简单的分类形式，结果只有两种。
- ◆ 例子：判断邮件是"垃圾邮件"还是"非垃圾邮件"；判断电影评论是"正面"还是"负面"。
- ◆ (2)多分类：结果是多个互斥类别中的一个。
- ◆ 例子：新闻分类，一篇报道只能属于"体育"、"财经"、"娱乐"中的一类。



5.5 应用案例

3. 常见框架

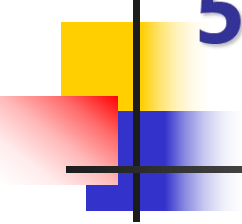
- ◆ (1)传统机器学习架构：基于人工特征工程和统计学习理论，适用于小数据场景。代表模型有：朴素贝叶斯、逻辑回归、SVM等。
- ◆ (2)词嵌入 + 浅层神经网络：将离散词映射为稠密向量，实现端到端学习。代表模型有：FastText、TextCNN、TextRNN和RCNN等。
- ◆ (3)注意力增强架构：引入注意力机制实现关键信息聚焦和长期依赖建模。代表模型有：HAN、Transformer和DPCNN等。
- ◆ (4)预训练语言模型架构：采用“大规模预训练 + 任务微调”范式，当前SOTA。BERT、RoBERTa、ALBERT和DistilBERT等。
- ◆ (5)长文本专用架构：解决BERT等模型输入长度限制问题。代表模型有：Longformer、BigBird、LED和Luna等。



5.5 应用案例

4. 常见数据集

- ◆ AG News (英文) : 主题范围广常被用作短文本分类的基准数据集, 在PyTorch等框架中可直接调用, 包括12.7万条新闻、4个类别。
- ◆ IMDb (英文) : 电影评论, 英文情感分析任务的标准基准之一, 每个类别有25000条高度平衡的数据, 包括5万条评论、2个类别 (正向/负向)。
- ◆ THUCNews (中文) : 由清华大学提供, 数据量大且分布均衡, 是中文新闻分类的经典大规模数据集, 包括约74万篇新闻、14个类别 (如体育、财经等)。
- ◆ 今日头条数据集 (中文) : 来自今日头条真实用户数据, 格式规范, 适合研究短文本新闻分类与标签预测, 包括约38万条、15个类别。
- ◆ ChnSentiCorp (中文) : 数据量适中, 是中文情感分析 (二分类) 入门最常用的数据集, 包括约7000条酒店评论、2个类别 (正向/负向)。



5.5 应用案例

5. 文本分类案例

【例5-3】设计一个基于GloVe预训练词向量和双向LSTM的文本情感分类系统，在IMDB影评数据集上实现情感二分类（正面/负面）。

1. 项目概述

- ◆ 这是一个基于 GloVe预训练词向量 和 双向LSTM 的IMDB影评情感二分类系统。采用模块化设计，支持独立训练、评估和预测。完全离线运行，在CPU上达到89-91%准确率，兼具高性价比和实用性的文本分类解决方案。

2. 项目目录结构

5.5 应用案例

5. 文本分类案例

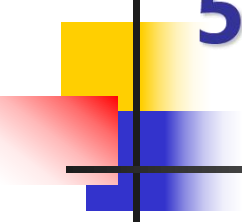
```
text_classification/
├── config.py           # 配置文件（所有超参数）
├── utils.py           # 工具函数（GloVe 加载、词汇表构建、NLTK 配置）
├── dataset.py         # 数据集类（数据加载和预处理）
├── model.py           # BiLSTM 模型定义
├── train.py           # 独立训练脚本
├── val.py             # 独立评估脚本
├── predict.py         # 独立预测脚本
├── nltk_data/         # NLTK 分词数据（本地）
│   ├── tokenizers/
│   │   └── punkt_tab/
├── data/              # 数据目录
│   ├── glove.6B.300d.txt # GloVe 词向量
│   └── IMDB_Dataset.csv  # IMDB 数据集
├── models/            # 模型保存目录
│   ├── best_bilstm_model.pt # 最佳模型
│   └── vocab.json         # 词汇表
├── logs/              # 训练日志
│   └── training_history.json
└── results/           # 评估结果
    ├── confusion_matrix.png
    └── roc_curve.png
```

5.5 应用案例

5. 文本分类案例

3. 模块文件

- ◆ (1)config.py: 配置中心, 主要功能包括: 统一管理所有路径(数据、模型、日志), 定义模型超参数(嵌入维度、隐藏层大小、层数等), 设置训练参数(批次大小、学习率、训练轮数), 配置设备(CPU/GPU)和随机种子。
- ◆ (2)utils.py: 工具函数库, 主要功能包括: NLTK配置(禁用自动下载, 使用本地分词数据), GloVe加载(读取预训练词向量文件(40万词, 300维)), 词汇表构建(统计词频, 过滤低频词, 计算GloVe覆盖率), 嵌入矩阵创建(将GloVe转换为PyTorch嵌入层权重), 随机种子设置(保证实验可复现性), 词汇表保存/加载: 持久化词到索引的映射。



5.5 应用案例

5. 文本分类案例

3. 模块文件

- ◆ (3)dataset.py : 数据处理, 主要包括:
- ◆ IMDBDataset类 (继承PyTorch Dataset, 实现数据读取) ;
- ◆ collate_fn函数 (自定义批处理, 堆叠成批张量) 。
- ◆ (4)model.py: BiLSTM模型, 主要包括:
- ◆ BiLSTMClassifier类: 双向LSTM分类器;
- ◆ save_model方法: 保存模型状态、配置、优化器、训练历史;
- ◆ load_model方法: 从检查点恢复完整模型。

5.5 应用案例

5. 文本分类案例

3. 模块文件

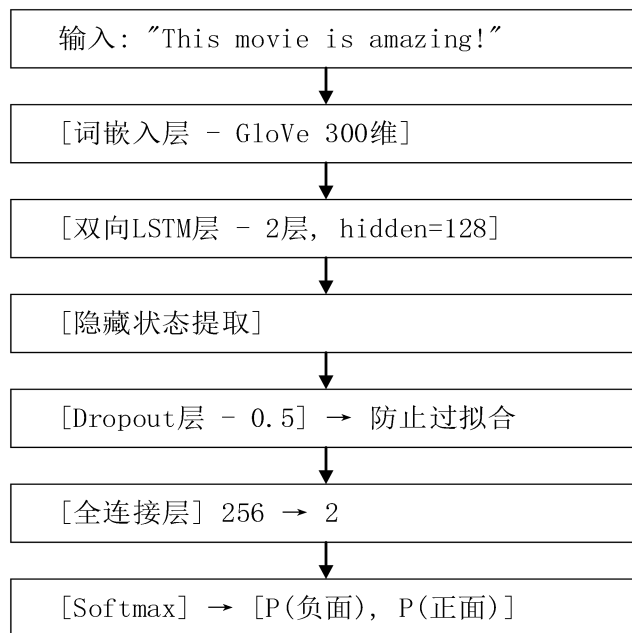
- ◆ (5)train.py: 训练模块, 主要包括包括: 自动加载IMDB数据集和GloVe词向量, 构建词汇表并创建嵌入矩阵, 训练BiLSTM模型, 支持早停和学习率调度, 自动保存最佳模型和词汇表。
- ◆ (6)val.py : 评估模块, 主要功能包括: 测试集上计算准确率、精确率、召回率、F1分数, 生成混淆矩阵可视化图, 绘制ROC曲线并计算AUC值, 输出详细的分类报告。
- ◆ (7)predict.py: 推理模块, 主要包括包括: 单条预测 (直接输入文本返回情感结果), 交互模式 (实时对话式预测), 批量预测 (从文件读取多条文本), 演示模式 (展示预设示例)

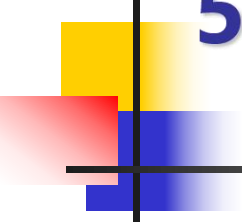
5.5 应用案例

5. 文本分类案例

4. 模型架构层次

- ◆ 项目的模型架构主要包括嵌入层、双向LSTM层、Dropout层和分类层，如图所示。



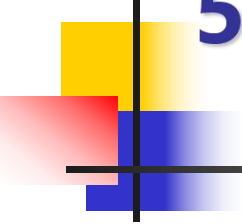


5.5 应用案例

5. 文本分类案例

5. 训练模型

- ◆ (1)使用默认配置
- ◆ `python train.py`
- ◆ (2)自定义参数
- ◆ `python train.py --epochs 20 --batch_size 32 --lr 0.001`

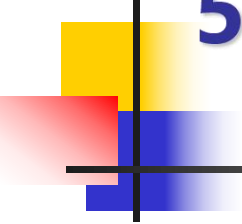


5.5 应用案例

5. 文本分类案例

5. 训练模型

- ◆ (3)完整参数
- ◆ `python train.py \`
- ◆ `--data_path data/IMDB_Dataset.csv \`
- ◆ `--glove_path data/glove.6B.300d.txt \`
- ◆ `--batch_size 64 \`
- ◆ `--epochs 10 \`
- ◆ `--max_len 300 \`
- ◆ `--save_dir models`

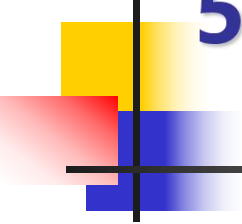


5.5 应用案例

5. 文本分类案例

6. 评估模型

- ◆ (1)默认配置
- ◆ `python val.py`
- ◆ (2)指定模型路径
- ◆ `python val.py --model_path models/best_bilstm_model.pt`
- ◆ (3)指定输出目录
- ◆ `python val.py --output_dir results`

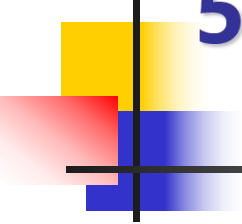


5.5 应用案例

5. 文本分类案例

7. 预测

- ◆ (1)单条预测
 - ◆ `python predict.py --text "This movie is amazing!"`
- ◆ (2)交互模式
 - ◆ `python predict.py --interactive`
- ◆ (3)演示模式
 - ◆ `python predict.py`
- ◆ (4)从文件读取
 - ◆ `python predict.py --file test.txt`

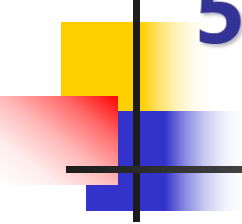


5.5 应用案例

5. 文本分类案例

7. 预测

- ◆ (5)运行结果
- ◆ 输入: "This movie is absolutely amazing!"
- ◆ 输出: 正面 (置信度: 0.96)
- ◆ 输入: "What a terrible waste of time."
- ◆ 输出: 负面 (置信度: 0.94)



5.5 应用案例

5. 文本分类案例

8. 改进方向

- ◆ (1)注意力机制：在LSTM后添加Self-Attention;
- ◆ (2)层次化模型：句子级→文档级（处理超长文本）；
- ◆ (3)模型集成：BiLSTM + TextCNN 投票；
- ◆ (4)数据增强：回译、同义词替换。